

Individual Assignment

Scenario 1: AI-Powered Knowledge Base Automation

Hesam Zoveidavian Poor – A22MJ4009

MJIIT

May 2026

1. Introduction

DHL logistics operations teams handle thousands of messages and document every day. These come from many different sources including chat messages from Microsoft Teams, long email threads, screenshots of errors, and plain text notes. All this information is important, but it is unstructured, scattered, and difficult to reuse. Because of this, creating clean and standardized Standard Operating Procedures (SOPs) becomes extremely slow and inconsistent.

This project delivers a prototype knowledge base system that automatically converts unstructured inputs into structured SOP articles and manages them through a controlled review workflow. The system is built using a Next.js web application backed by a JSON file database, integrated with a UiPath RPA bot that fetches files from Google Drive and an OpenAI language model that generates clean structured content from raw messy text.

2. Scenario Description

The chosen scenario is Scenario 1, AI-Powered Knowledge Base Automation, as described in the assignment brief. DHL aims to automate the transformation of messy operational inputs into clean knowledge articles. Typical inputs include chat screenshots from Microsoft Teams, plain text notes with operational instructions, PDF reports, and Word documents shared through Google Drive.

The system supports multiple input types and guides users through a draft, review, and publish workflow. A UiPath RPA bot monitors a designated Google Drive folder and automatically downloads new files, processes their content through OCR or text extraction, sends the extracted text to OpenAI, and posts the resulting structured article to the web application without any human involvement.

3. Project Objectives

The objectives of this project are derived from the functional requirements in the assignment specification:

- a. Build a secured web application with JWT-based login and role-based access for admins and editors. The application must provide an upload page where users can paste raw text or upload files, generate a structured SOP draft using OpenAI, allow editing, and save the result as a Draft article.
- b. Manage articles through a controlled status workflow where articles progress from Draft to Reviewed to Published. Editors can create and review articles but cannot publish or delete. Admins have full control. The dashboard supports search and filtering by keyword, status, tag, creator, and date, and displays a statistics panel and tags cloud.
- c. Expose a REST API through Next.js API routes so all data in db.json can be accessed through standard HTTP methods. The API supports GET, POST, PATCH, and DELETE on articles, users, duplicate hashes, logs, and statistics.
- d. Integrate with UiPath so the bot can fetch files from Google Drive, perform duplicate detection using MD5 hashing with a 14-day window, extract text using OCR or PDF reading, generate structured SOPs via the API, create articles, update statuses, handle errors with screenshots, and write a summary run report.
- e. Use OpenAI GPT-4o Mini as the language model to convert raw messy text into structured articles with a clear title, a 2 to 3 sentence summary, numbered steps, relevant tags, and a category.

4. System Architecture

The system consists of three main components:

Component	Technology	Responsibilities
Web Application	Next.js 14 TypeScript App Router	Login, dashboard, upload, article management, all REST API endpoints, read and write db.json
Database	db.json flat file	Stores users, articles, duplicate hashes, and run logs
RPA Bot	UiPath Studio	Fetches from Google Drive, processes files, creates articles via API, generates reports
AI Layer	OpenAI GPT-4o Mini	Converts raw text into structured SOP with title, summary, steps, tags, category

Table 4.1 System Architecture of the DHL Knowledge Base System

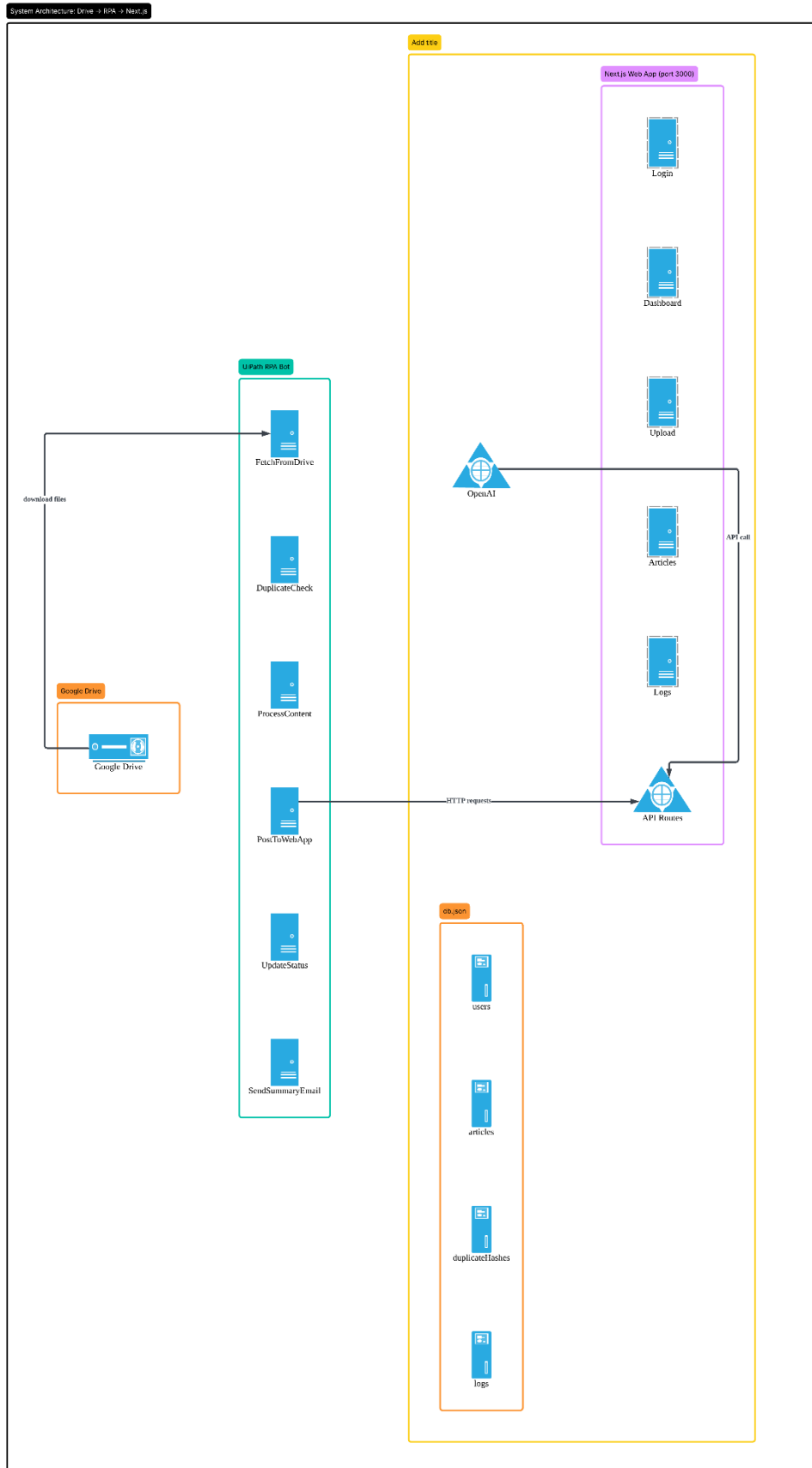


Figure 4.1 System Architecture of the DHL Knowledge Base System

4.1. Data Flow

- a. The UiPath bot logs into the web application by sending a POST request to `/api/auth` with the uipath-bot credentials and extracts a JWT token from the response using a regular expression.
- b. `FetchFromDrive.xaml` connects to Google Drive using a Google service account, lists all files in the DHL-RPA folder, and downloads them to `C:\DHL_Temp`. File paths are written to `C:\DHL_Temp\filelist.txt` for the main workflow to read.
- c. For each file, `DuplicateCheck.xaml` generates an MD5 hash of the file content and sends a GET request to `/api/duplicates`. If the same hash was processed within the last 14 days the file is skipped and the duplicate counter increases. The result is written to `C:\DHL_Temp\dupcheck.txt`.
- d. `ProcessContent.xaml` determines the file type by extension. PDF files use the Read PDF Text activity. Image files such as JPG and PNG use Tesseract OCR. All other files are read as plain text. The extracted text is sent as a POST request to `/api/process`.
- e. The `/api/process` route in `Next.js` cleans the text of control characters, sends it to OpenAI GPT-4o Mini with a structured prompt, and returns a JSON object with title, summary, steps, tags, and category. `ProcessContent.xaml` writes this JSON to `C:\DHL_Temp\structured.txt`.
- f. `PostToWebApp.xaml` reads `structured.txt` and sends a POST request to `/api/articles` to create the article in `db.json` with `createdBy` set to uipath-bot and status set to Draft.
- g. `UpdateStatus.xaml` sends a PATCH request to the article endpoint to confirm the status and add a processing note with the date and time.
- h. After all files are processed `SendSummaryEmail.xaml` builds a summary showing articles created, duplicates skipped, and errors. It writes this to `C:\DHL_Logs\summary.txt` and posts it to `/api/logs`.

4.2. Deployment

The project runs entirely on a local machine. To start the system:

- a. Install dependencies by running `npm install` inside the web-app folder.
- b. Create `.env.local` with `JWT_SECRET` and `OPENAI_API_KEY`.
- c. Run `npm run dev` to start the Next.js application on port 3000.
- d. Upload sample files to the DHL-RPA folder in Google Drive.
- e. Open UiPath Studio, open the DHL_RPA project, and press F5.

5. API and JSON Structure

All data is stored in db.json. Next.js API routes read and write this file directly using Node.js file system operations. No separate database server is required.

Method and Endpoint	Description
POST /api/auth	Verifies username and password against users in db.json. Returns JWT token.
GET /api/articles	Returns articles. Supports search, status, tag, creator, date query parameters.
POST /api/articles	Creates a new article. Sets status to Draft and records createdBy and timestamps.
PATCH /api/articles/:id	Updates article fields. Records status changes in statusHistory array. Enforces role restrictions.
DELETE /api/articles/:id	Deletes an article. Returns 403 if user is not admin.
POST /api/process	Receives rawText, cleans control characters, calls OpenAI, returns structured SOP JSON.
GET /api/duplicates	Checks if a hash exists in duplicateHashes within 14 days. Returns isDuplicate true or false.
POST /api/duplicates	Registers a new hash after successful processing.
GET /api/stats	Returns counts of total, draft, reviewed, published, byBot, byHuman.
GET /api/logs	Returns all run log entries.
POST /api/logs	Appends a new log entry posted by UiPath after each run.

Table 5.1: API Methods and Endpoint

5.1. Article Schema

Each article in db.json contains these fields:

- id: UUID generated at creation time.
- title, summary, steps, tags, category: Core SOP content generated by OpenAI.
- status: Current workflow state. Draft, Reviewed, or Published.
- statusHistory: Array of entries each with status, changedAt, by, and optional note.
- createdBy: Username of creator. Shows uipath-bot for automated articles.
- source: Origin of content. Values include uipath-drive or manual-upload.
- rawInput: The original unprocessed text kept for reference.
- createdAt, updatedAt: ISO 8601 timestamps.

The duplicateHashes collection stores hash and checkedAt for each processed file to support the 14-day duplicate window.

6. User Interface

The web application provides a clean interface for creating and managing knowledge articles with role-based access enforced on both the frontend and the API.

6.1. Login Page

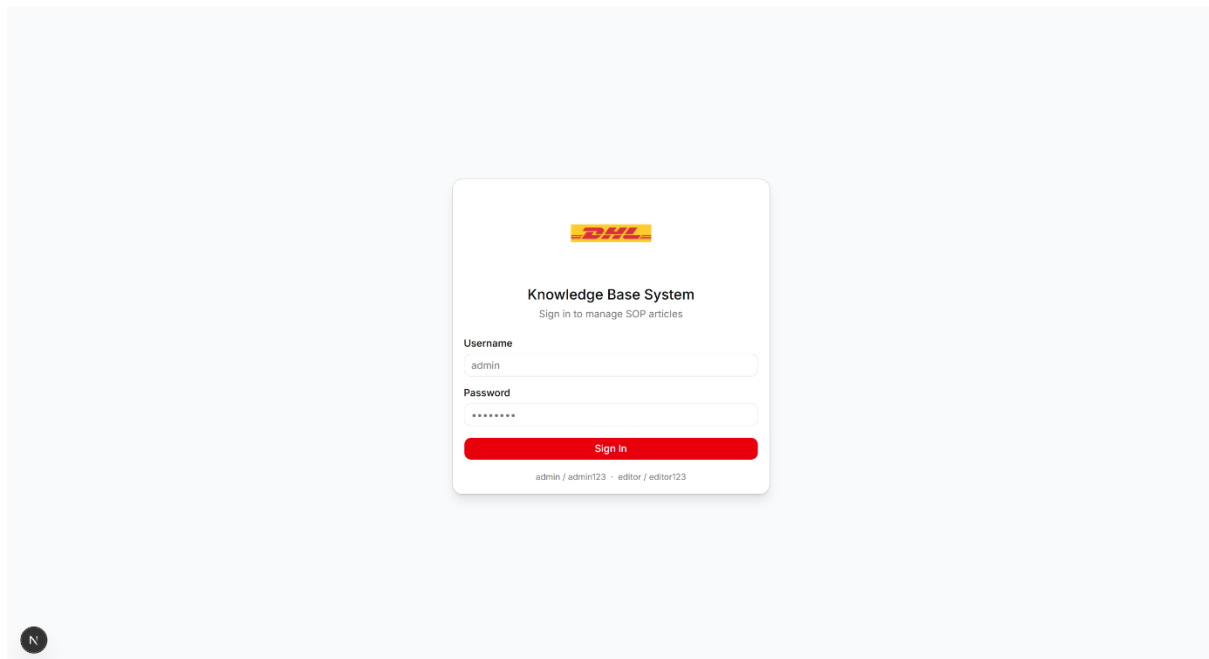


Figure 6.1 Login Page

The login page requires a username and password which are verified against the users array in `db.json`. A JWT token is returned on success and stored in `localStorage`. The Next.js middleware intercepts all requests and redirects unauthenticated users to the login page. The DHL logo is displayed at the top of the form.

6.2. Dashboard

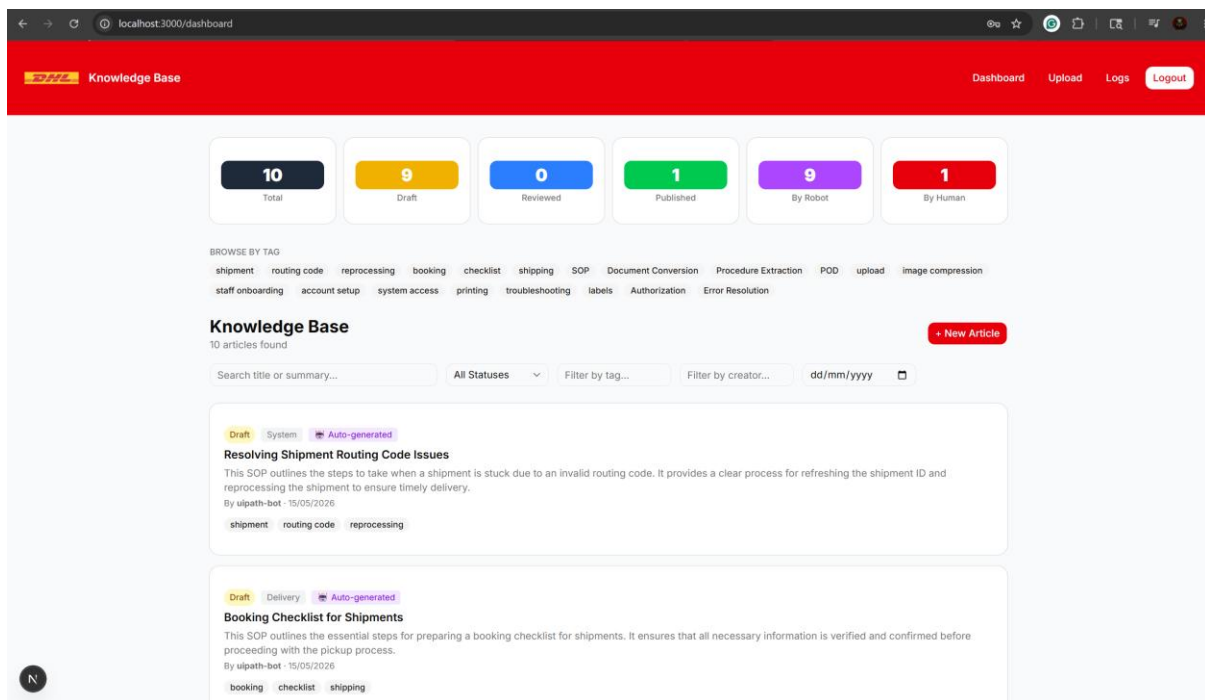


Figure 6.2 Dashboard with Statistics and Article List

The dashboard displays real-time statistics including total articles, draft count, reviewed count, published count, articles created by the UiPath robot, and articles created by human users. A clickable tags cloud allows one-click filtering by topic. Users can search by keyword and filter by status, tag, creator, and date. A green banner appears automatically when UiPath has created new articles in the last five minutes.

6.3. Article Status Workflow

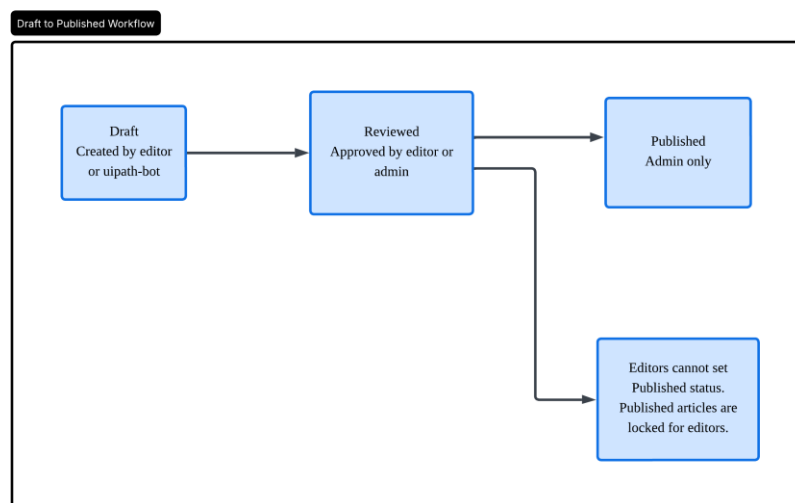
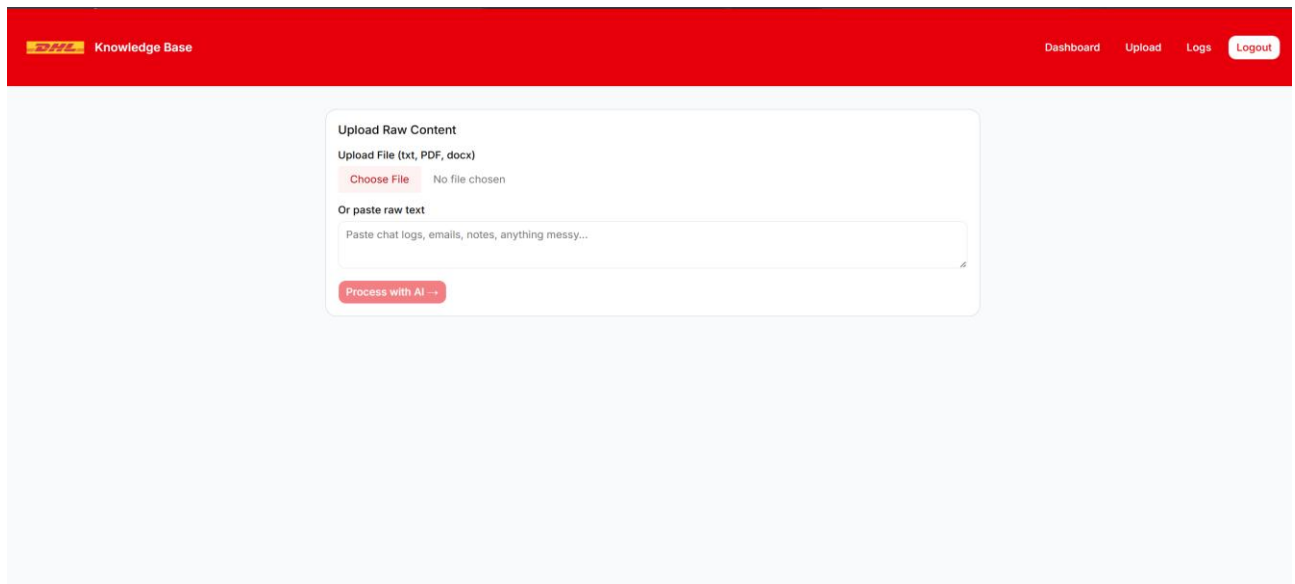


Figure 6.3 Article Status Workflow

Articles follow a three-stage workflow as shown in Figure 6.3. Articles start as Draft when created manually or by the UiPath bot. An editor or admin can move the article to Reviewed. Only an admin can move an article to Published. Editors cannot set the Published status and cannot modify articles that are already published. Every status change is recorded in the statusHistory array with a timestamp and the username of who made the change.

6.4. Upload Page



The screenshot shows a web interface for uploading raw content. At the top is a red header bar with the 'DHL' logo and 'Knowledge Base' text on the left, and 'Dashboard', 'Upload', 'Logs', and a 'Logout' button on the right. The main content area is light blue and contains a white rounded rectangle titled 'Upload Raw Content'. Inside this rectangle, there is a section 'Upload File (txt, PDF, docx)' with a 'Choose File' button and the text 'No file chosen'. Below this is a section 'Or paste raw text' with a large text area containing the placeholder text 'Paste chat logs, emails, notes, anything messy...'. At the bottom of the white rectangle is a red button labeled 'Process with AI -->'.

Figure 6.4 Upload Page for Creating SOP Articles

The upload page allows staff to submit raw content manually. Users paste unstructured text into the textarea or upload a text file. When Process with AI is clicked, the text is sent to `/api/process` which calls OpenAI and returns a structured draft. The DraftBuilder component shows the generated title, summary, steps, and tags and allows the user to edit any field before saving. The article is saved as Draft.

6.5. Article Detail Page

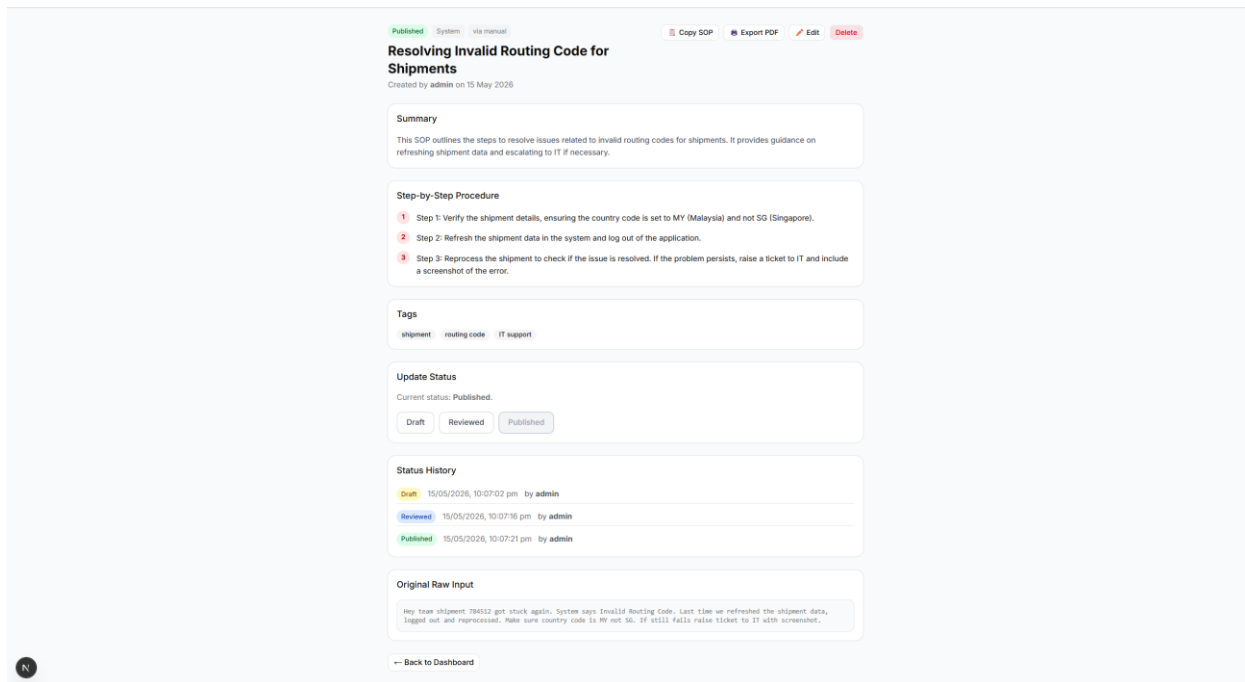


Figure 6.5 Article Detail Page with Status History and Audit Trail

The article detail page shows the full SOP including summary, numbered steps, tags, category, source, and creation details. Admins can edit the content directly, copy the SOP to clipboard, or export as PDF using the browser print function. The status update section shows only the transitions allowed for the current user role. The status history at the bottom shows every change as a full audit trail with timestamps and usernames.

6.6. Logs Page



Figure 6.6 UiPath Run Logs Page

The logs page displays all UiPath automation run reports posted to /api/logs after each run. Each entry shows the timestamp and the full summary including articles created, duplicates skipped, and errors. This allows managers to monitor automation activity directly from the web application.

7. RPA Workflow Explanation

The UiPath project is called DHL_RPA and consists of six separate workflow files. Each file has a single responsibility making the automation modular and easier to debug. Using separate workflows instead of one large file also means any individual workflow can be updated or replaced without affecting the rest of the system.

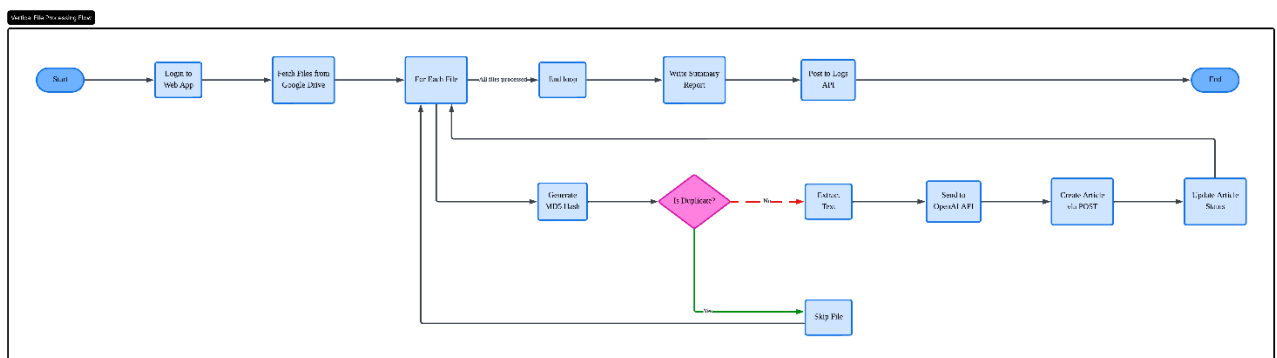


Figure 7.1 UiPath RPA Automation Workflow

Workflow	Purpose
Main.xaml	Orchestrator. Logs in to get JWT token, reads filelist.txt, loops through each file with Try-Catch, calls all sub-workflows, handles errors.
FetchFromDrive.xaml	Connects to Google Drive via service account. Downloads files from DHL-RPA to C:\DHL Temp. Writes all paths to filelist.txt.
DuplicateCheck.xaml	Generates MD5 hash of file content. Calls GET /api/duplicates. Writes true or false to dupcheck.txt.
ProcessContent.xaml	Checks file extension. PDFs use Read PDF Text. Images use Tesseract OCR with scale 3. Others use Read Text File. Sends text to /api/process. Writes response to structured.txt.
PostToWebApp.xaml	Reads structured.txt. Sends POST to /api/articles. Sets createdBy to uipath-bot.
UpdateStatus.xaml	Sends PATCH to confirm Draft status and add processing timestamp note.
SendSummaryEmail.xaml	Builds summary string with created, duplicate, and error counts. Writes to C:\DHL Logs\summary.txt. Posts to /api/logs.

Table 7.1: Workflows and their Respective Purpose

Main.xaml uses a text file intermediary approach because the newer version of UiPath Studio does not reliably pass Out arguments between invoked workflows. DuplicateCheck writes its result to dupcheck.txt which Main reads back as a string and converts to a Boolean. ProcessContent writes the structured JSON to structured.txt which PostToWebApp reads directly using System.IO.File.ReadAllText. This approach ensures reliable data passing across all workflow boundaries.

The Try-Catch block inside the For Each loop ensures that if any single file fails, the error is logged with a message, a screenshot is saved to C:\DHL_Logs, the error counter increments, and the loop continues with the next file.

8. Testing and Validation

The system was tested with nine sample files in the Google Drive DHL-RPA folder including five plain text files with operational notes, two Word documents, and two Teams chat screenshots as JPG images.

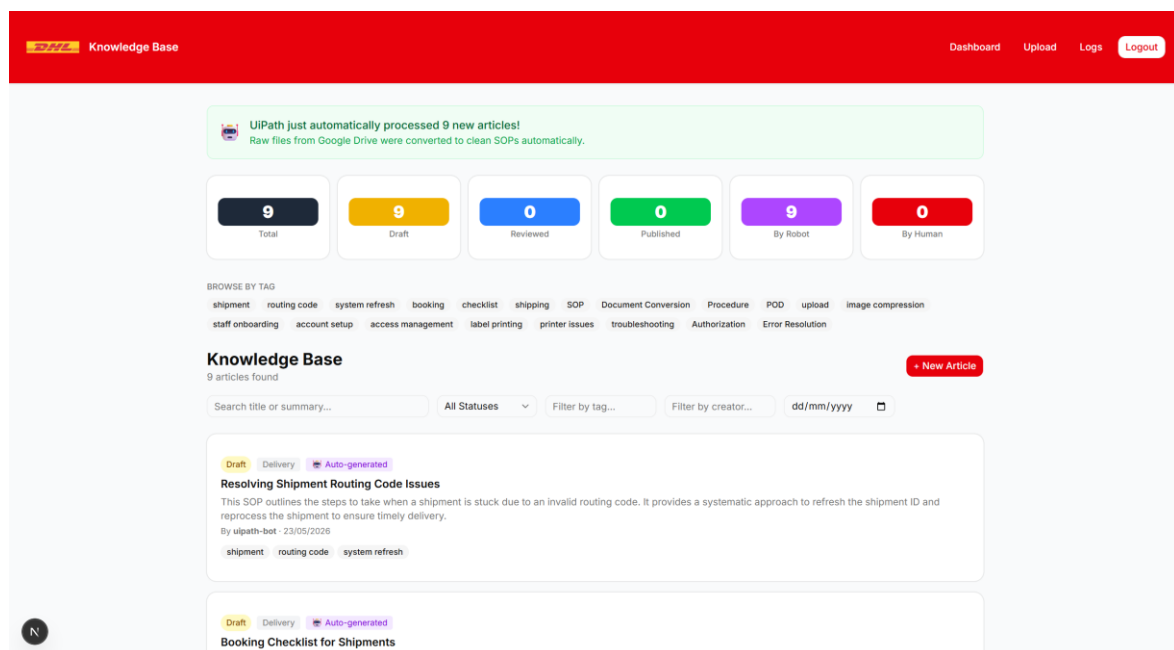
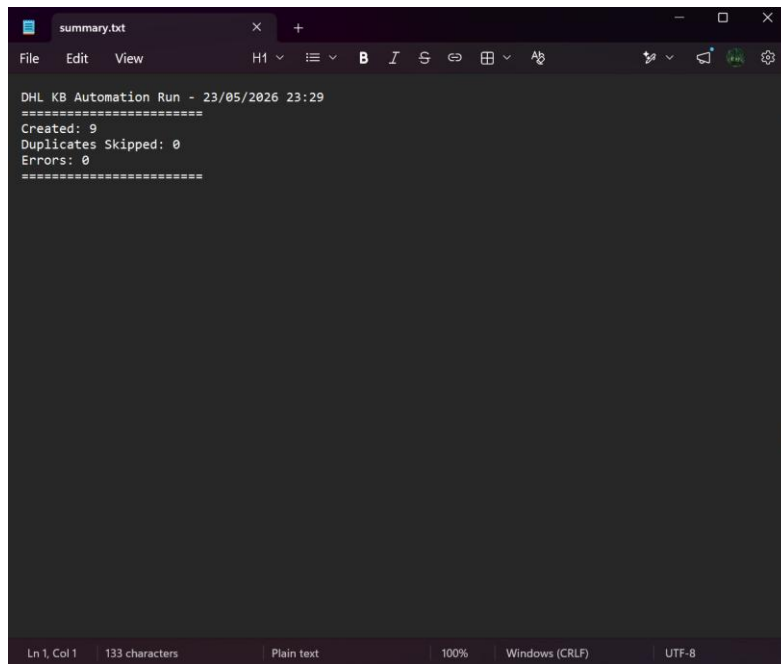


Figure 8.1 Dashboard After First UiPath Run Showing 9 Auto-Generated Articles



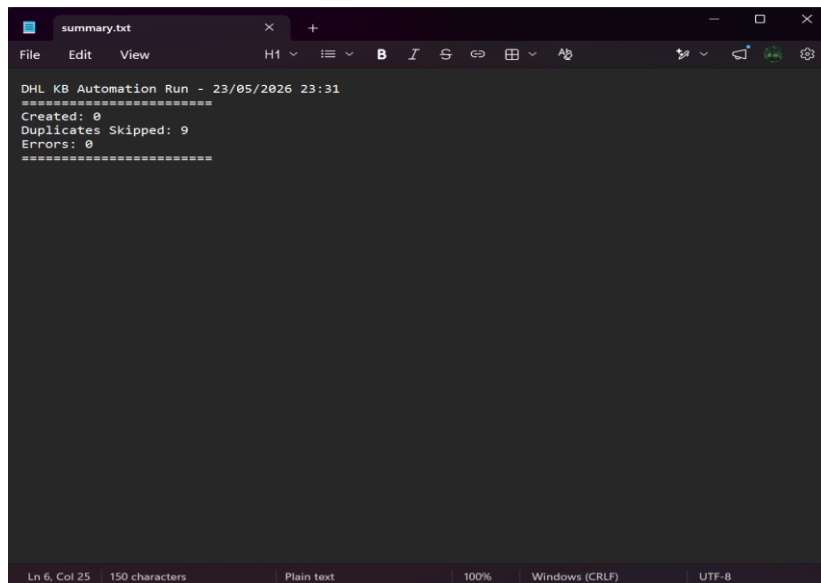
```
summary.txt
File Edit View H1 B I S 100% Windows (CRLF) UTF-8

DHL KB Automation Run - 23/05/2026 23:29
=====
Created: 9
Duplicates Skipped: 0
Errors: 0
=====

Ln 1, Col 1 | 133 characters | Plain text | 100% | Windows (CRLF) | UTF-8
```

Figure 8.2 Summary Report After First Run

On the first run the bot connected to Google Drive, downloaded all nine files, processed each one, and created nine draft articles. The run report showed Created: 9, Duplicates Skipped: 0, Errors: 0. All articles were tagged as Auto-generated and showed uipath-bot as the creator.



```
summary.txt
File Edit View H1 B I S 100% Windows (CRLF) UTF-8

DHL KB Automation Run - 23/05/2026 23:31
=====
Created: 0
Duplicates Skipped: 9
Errors: 0
=====

Ln 6, Col 25 | 150 characters | Plain text | 100% | Windows (CRLF) | UTF-8
```

Figure 8.3 Summary Report After Second Run Showing Duplicate Detection

When the workflow was run again immediately with the same files, all nine were correctly identified as duplicates and skipped. The run report showed Created: 0, Duplicates Skipped: 9, Errors: 0.

Role-based access was verified by logging in as editor and attempting to publish and delete articles. The Published button was hidden in the UI and the `/api/articles/:id DELETE` endpoint returned 403 Forbidden. Admin login allowed full control including publishing and deletion.

9. Limitations and Future Improvements

This system is a local prototype. The web application uses a flat JSON file as the database which works for demonstration but would not scale to thousands of articles in production. A real deployment would use a relational or document database such as PostgreSQL or MongoDB.

OCR quality for Teams chat screenshots is limited by Tesseract accuracy on dark-themed interfaces. The raw text extracted from images often contains noise but OpenAI is able to understand the intent and generate a clean SOP regardless. A future improvement would be to use a cloud OCR service for better accuracy.

Word documents in DOCX format cannot be read as plain text by UiPath without the Word Application Scope activity which requires Microsoft Word installed. The current system uses a fallback message for DOCX files. A future version could extract DOCX content server-side using a library such as mammoth in the Next.js API.

Email notifications are not implemented because no SMTP server was available during development. The summary report is saved to a local text file and posted to /api/logs instead. A production version could use UiPath Send SMTP Mail Message or a transactional email API.

10. Conclusion

This project fulfills all mandatory requirements in the assignment brief. The Next.js web application provides JWT-secured role-based access, supports manual text upload with AI processing, manages SOP articles through a Draft to Reviewed to Published workflow, and exposes a complete REST API. The UiPath RPA bot automatically ingests files from Google Drive, performs duplicate detection using MD5 hashing with a 14-day window, processes content using Tesseract OCR and PDF reading, creates articles via HTTP requests, handles errors with screenshots, and generates run reports.

The optional LLM component was implemented using OpenAI GPT-4o Mini which converts raw messy text including OCR output into clean structured SOPs with professional titles, summaries, numbered steps, and relevant tags. The system demonstrates a complete automation pipeline from raw unstructured input to a managed published knowledge article.